

BLG483E – Homework 3

Local Wikipedia RAG Assistant: Technical Report and Code Walkthrough

Name / Student ID: [Uygar GÖK / 820220336]

May 4, 2026

Abstract

This report documents a fully *local* Retrieval-Augmented Generation (RAG) assistant that answers questions about famous people and places using Wikipedia-derived text. The system ingests English Wikipedia extracts, splits them into overlapping chunks, embeds them with a local `sentence-transformers` model, stores vectors in a persistent Chroma database with metadata, routes queries to `person` and/or `place` scopes, and generates answers with a local Ollama LLM without any external LLM API. The report explains each source file, the end-to-end execution path, and representative code fragments that illustrate how the implementation works.

Contents

1	Introduction and assignment goals	3
2	Repository layout and module responsibilities	3
3	End-to-end system architecture	4
3.1	Runtime artifacts on disk	4
4	Configuration layer (<code>config.py</code>)	4
5	Entity inventory (<code>entities.py</code>)	5
6	Chunk schema (<code>chunk_row.py</code>)	5
7	Chunking strategy (<code>chunking.py</code>)	5
8	Wikipedia ingestion and SQLite persistence (<code>wikipedia_ingest.py</code>)	6
8.1	API call and robustness	6
8.2	SQLite schema	6
8.3	Corpus builder output	7
9	Vector store: embeddings and Chroma (<code>vector_store.py</code>)	7
9.1	Class <code>LocalVectorStore</code>	7
9.2	Upsert pipeline	7
9.3	Metadata-filtered vector query	7
9.4	Resilience: reset and index corruption	8

10 Retrieval and query routing (retrieval.py)	8
10.1 Rule-based classification (classify_query)	8
10.2 Primary retrieval (retrieve_context)	8
10.3 SQLite fallback retrieval	8
11 Local answer generation (generation.py)	8
12 Command-line interface (main.py)	9
13 One-shot ingestion script (ingest.py)	9
14 How to run (reproducible checklist)	10
15 Design decisions and trade-offs	10
16 Limitations and known failure modes	10
17 Conclusion	11

1 Introduction and assignment goals

The course homework (BLG483E HW3) requires building a simplified ChatGPT-like system that:

- runs entirely on the student’s machine (localhost);
- ingests Wikipedia pages for at least 20 people and 20 places, including a mandatory minimum set;
- defines a chunking strategy for large documents;
- uses *local* embeddings and a vector database;
- classifies whether a question targets a person, a place, or both, and retrieves accordingly;
- uses a *local* language model to answer from retrieved context and to say **I don’t know** when the context is insufficient;
- provides a simple chat UI or CLI.

This implementation satisfies these requirements using Python, SQLite for structured ingestion records, Chroma for vector search, SentenceTransformers for embeddings, and Ollama for local generation.

2 Repository layout and module responsibilities

Table 1 summarizes the main Python modules and their roles.

File	Responsibility
<code>config.py</code>	Central configuration: filesystem paths, Wikipedia API URL and headers, embedding model id, Ollama model id, chunk size/overlap, and retrieval TOP_K.
<code>entities.py</code>	Curated lists PEOPLE and PLACES (each at least 20 entries), including the homework-required minimum entities.
<code>chunk_row.py</code>	TypedDict describing one chunk row passed from ingestion to the vector store.
<code>chunking.py</code>	Deterministic sliding-window chunking with overlap on normalized plaintext.
<code>wikipedia_ingest.py</code>	Fetches Wikipedia extracts, persists docs/chunks in SQLite, returns a list of ChunkRow objects for embedding.
<code>vector_store.py</code>	Chroma persistent client, embedding, upsert, metadata-filtered query, and resilient reset.
<code>retrieval.py</code>	Rule-based query classification, Chroma retrieval, and SQLite keyword fallback if Chroma fails.
<code>generation.py</code>	Ollama chat completion with a strict system prompt for grounded answers.
<code>main.py</code>	Interactive CLI loop: commands and question answering with optional source printing.
<code>ingest.py</code>	One-shot pipeline for demo/reset: reset store, verbose ingest, upsert to Chroma.

Table 1: Main Python modules and responsibilities.

Additional deliverable documents (not Python) include `README.md`, `Product_prd.md`, and `recommendation.md`.

3 End-to-end system architecture

Figure 1 shows the logical pipeline from ingestion to answer generation.

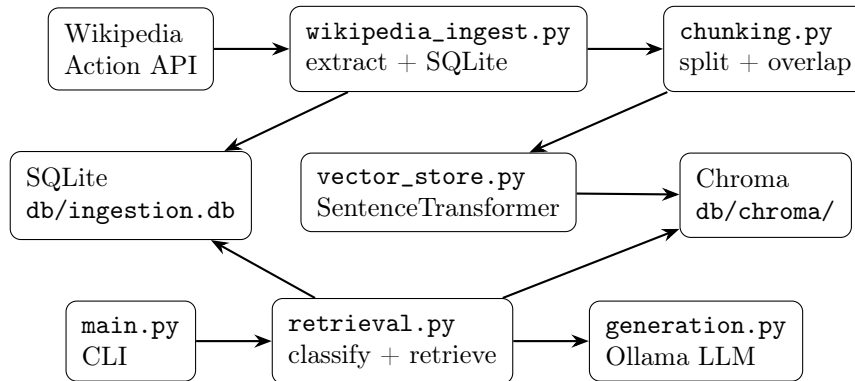


Figure 1: High-level data flow: Wikipedia → SQLite chunks → embeddings → Chroma; user query → retrieval (Chroma with optional SQLite fallback) → Ollama.

3.1 Runtime artifacts on disk

- `db/ingestion.db`: SQLite tables `docs` (one row per Wikipedia page) and `chunks` (one row per chunk), enabling provenance and fallback retrieval.
- `db/chroma/`: Chroma persistent vector index and metadata for approximate nearest-neighbor search.
- `data/`: reserved directory for auxiliary local files (if needed).

4 Configuration layer (`config.py`)

All “magic constants” live in `config.py` so the rest of the codebase stays readable and reproducible. Listing 1 shows the essential settings.

Listing 1: Central paths and model identifiers (`config.py`).

```
1 from pathlib import Path
2
3 BASE_DIR = Path(__file__).resolve().parent
4 DATA_DIR = BASE_DIR / "data"
5 DB_DIR = BASE_DIR / "db"
6 CHROMA_DIR = DB_DIR / "chroma"
7 SQLITE_PATH = DB_DIR / "ingestion.db"
8
9 EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
10 OLLAMA_MODEL = "llama3.2:3b"
11
12 CHUNK_SIZE = 750
13 CHUNK_OVERLAP = 120
14 TOP_K = 5
15
16 WIKIPEDIA_API = "https://en.wikipedia.org/w/api.php"
17 WIKIPEDIA_HEADERS = {
18     "User-Agent": "BLG483E-LocalWikipediaRAG/1.0 (course homework; Python requests)",
19     "Accept": "application/json",
20 }
```

Why WIKIPEDIA_HEADERS? Wikimedia commonly rejects anonymous `requests` calls without a descriptive `User-Agent`. Without it, ingestion may fail with HTTP 403, producing empty chunks.

5 Entity inventory (`entities.py`)

The homework requires at least 20 people and 20 places, including a fixed minimum subset. The project encodes these as two Python lists, `PEOPLE` and `PLACES`. During ingestion, each title is paired with a label `entity_type` $\in$ `{person, place}`, which later drives metadata filtering during retrieval.

6 Chunk schema (`chunk_row.py`)

Chunks are the atomic units stored in Chroma. A `ChunkRow` is a `TypedDict` ensuring consistent keys and value types between ingestion and `vector_store.py`:

Listing 2: Chunk record schema (`chunk_row.py`).

```
1 from typing import TypedDict
2
3 class ChunkRow(TypedDict):
4     id: str
5     title: str
6     entity_type: str
7     source_url: str
8     chunk_index: int
9     text: str
```

The `id` is a deterministic string key such as `person:Albert Einstein:0`, which makes upserts idempotent and debuggable.

7 Chunking strategy (`chunking.py`)

Wikipedia extracts can be very long. The chunker:

1. collapses whitespace to single spaces (normalization);
2. slides a window of length `CHUNK_SIZE` characters;
3. advances the window by `step = max(1, CHUNK_SIZE - CHUNK_OVERLAP)` so consecutive chunks overlap by `CHUNK_OVERLAP` characters.

Listing 3: Sliding-window chunking with overlap (`chunking.py`).

```
1 def chunk_text(text: str, chunk_size: int, overlap: int) -> List[str]:
2     cleaned = " ".join(text.split())
3     if not cleaned:
4         return []
5
6     chunks: List[str] = []
7     start = 0
8     step = max(1, chunk_size - overlap)
9     while start < len(cleaned):
10         end = start + chunk_size
11         chunk = cleaned[start:end].strip()
12         if chunk:
13             chunks.append(chunk)
14         start += step
15     return chunks
```

Design rationale. Overlap reduces the chance that an important sentence sits on a chunk boundary and becomes weakly represented in both neighboring chunks for embedding and retrieval.

8 Wikipedia ingestion and SQLite persistence (wikipedia_ingest.py)

8.1 API call and robustness

The module calls the Wikipedia query API with `prop=extracts` and `explaintext=1` to retrieve plain-text extracts (not HTML). Network failures and transient HTTP errors are handled with retries and exponential backoff-style sleep for status codes like 429 and certain 5xx responses.

Listing 4: Wikipedia fetch with retries (wikipedia_ingest.py, excerpt).

```
1 def fetch_wikipedia_extract(title: str) -> Tuple[str, str]:
2     params = {
3         "action": "query",
4         "format": "json",
5         "prop": "extracts",
6         "explaintext": 1,
7         "redirects": 1,
8         "titles": title,
9     }
10    for attempt in range(6):
11        try:
12            response = requests.get(
13                WIKIPEDIA_API, params=params, headers=WIKIPEDIA_HEADERS, timeout=30
14            )
15            response.raise_for_status()
16            data = response.json()
17            pages = data["query"]["pages"]
18            page = next(iter(pages.values()))
19            extract = page.get("extract", "")
20            page_title = page.get("title", title)
21            source_url = f"https://en.wikipedia.org/wiki/{page_title.replace(' ', '_')}"
22            return extract, source_url
23        except requests.HTTPError as exc:
24            status = exc.response.status_code if exc.response is not None else None
25            if status not in (429, 500, 502, 503, 504) or attempt == 5:
26                raise
27            time.sleep(1.0 * (attempt + 1))
28        except requests.RequestException as exc:
29            if attempt == 5:
30                raise
31            time.sleep(1.0 * (attempt + 1))
```

8.2 SQLite schema

Two tables are maintained:

- **docs**: one row per ingested page with `title` (primary key), `entity_type`, canonical `source_url`, and `text_length`.
- **chunks**: one row per chunk with stable `id`, foreign linkage via `title + entity_type`, `chunk_index`, and `text`.

At the start of each full ingest, the implementation clears prior SQLite rows so a failed partial ingest cannot leave misleading “person-only” residue when places were skipped.

8.3 Corpus builder output

`build_local_corpus` returns a flat list of `ChunkRow` dictionaries suitable for immediate embedding and upsert. Optional `verbose=True` prints per-entity progress and a final success/failure summary for demos and debugging.

9 Vector store: embeddings and Chroma (`vector_store.py`)

9.1 Class `LocalVectorStore`

On construction, the class:

- ensures `CHROMA_DIR` exists;
- creates a `chromadb.PersistentClient` pointing at that directory;
- opens (or creates) a single collection named `wiki_chunks`;
- loads `SentenceTransformer(EMBEDDING_MODEL)` once (this can dominate startup time).

9.2 Upsert pipeline

`upsert_rows` embeds all chunk texts in batch, builds parallel lists of `ids`, `embeddings`, `documents`, and `metadatas`, then calls Chroma `collection.upsert`. Metadata includes `entity_type` for homework-compliant filtering.

Listing 5: Embedding batch and upsert (`vector_store.py`, excerpt).

```
1 def upsert_rows(self, rows: List[ChunkRow]) -> None:
2     if not rows:
3         return
4     texts = [r["text"] for r in rows]
5     embeddings = self.embedder.encode(texts, show_progress_bar=True).tolist()
6     ids = [r["id"] for r in rows]
7     metadatas: List[Metadata] = [
8         cast(Metadata, {
9             "title": r["title"],
10            "entity_type": r["entity_type"],
11            "source_url": r["source_url"],
12            "chunk_index": r["chunk_index"],
13        })
14     for r in rows
15 ]
16 try:
17     self.collection.upsert(
18         ids=ids, embeddings=embeddings, documents=texts, metadatas=metadatas,
19     )
20 except Exception:
21     self.reset()
22     self.collection.upsert(
23         ids=ids, embeddings=embeddings, documents=texts, metadatas=metadatas,
24     )
```

9.3 Metadata-filtered vector query

The homework requires retrieving from the correct “store” or applying filtering. This project uses **Option B**: one collection with `entity_type` metadata. The query builds a Chroma `where` clause:

- single filter: `{"entity_type": "person"} or {"entity_type": "place"}`;
- combined filter: `{"$or": [{"entity_type": "person"}, {"entity_type": "place"}]}`.

9.4 Resilience: reset and index corruption

Some environments can corrupt on-disk HNSW segments. `reset()` attempts a normal collection delete/recreate, and if the collection is still unreadable it deletes the entire `CHROMA_DIR` and reinitializes the client. `upsert_rows` retries once after `reset()` on upsert failure.

10 Retrieval and query routing (`retrieval.py`)

10.1 Rule-based classification (`classify_query`)

The classifier is intentionally simple (homework-allowed): it checks whether the query contains known entity titles from `PEOPLE/PLACES`, and also checks small keyword hint sets (e.g., “where” suggests a place, “who” suggests a person). If both signals appear, retrieval widens to both entity types.

Listing 6: Query routing logic (`retrieval.py`, excerpt).

```
1 def classify_query(query: str) -> List[str]:
2     q = query.lower()
3     has_person_name = any(name.lower() in q for name in PEOPLE)
4     has_place_name = any(name.lower() in q for name in PLACES)
5     has_person_hint = any(word in q for word in PERSON_HINTS)
6     has_place_hint = any(word in q for word in PLACE_HINTS)
7
8     if (has_person_name and has_place_name) or (has_person_hint and has_place_hint):
9         return ["person", "place"]
10    if has_person_name or has_person_hint:
11        return ["person"]
12    if has_place_name or has_place_hint:
13        return ["place"]
14    return ["person", "place"]
```

10.2 Primary retrieval (`retrieve_context`)

`retrieve_context` calls `store.query` with the user question as the query text, embedding it with the same model used at ingest time (required for meaningful ANN search). Returned documents are concatenated into a numbered context string for the LLM.

10.3 SQLite fallback retrieval

If Chroma raises `RuntimeError` (unreadable index), the module falls back to SQLite: it loads candidate chunks filtered by `entity_type`, scores them with a lightweight token overlap heuristic between query and chunk text (plus a title match boost), and returns the top-`TOP_K` chunks. This preserves demo functionality even when the vector index is unstable.

11 Local answer generation (`generation.py`)

The generator uses the Ollama Python client with a low temperature. The system prompt forces grounding: answers must come only from the provided context; if insufficient, the model should output I don’t know. The code also short-circuits empty context and handles Ollama failures safely.

Listing 7: Grounded generation with Ollama (`generation.py`).

```
1 def generate_answer(query: str, context: str) -> str:
2     if not context.strip():
3         return "I don't know"
```



```

4
5     prompt = (
6         f"Context:\n{context}\n\n"
7         f"Question: {query}\n"
8         "Answer using only context."
9     )
10    try:
11        response = ollama.chat(
12            model=OLLAMA_MODEL,
13            messages=[
14                {"role": "system", "content": SYSTEM_PROMPT},
15                {"role": "user", "content": prompt},
16            ],
17            options={"temperature": 0.1},
18        )
19    except Exception:
20        return "I don't know"
21
22    answer = response["message"]["content"].strip()
23    return answer or "I don't know"

```

12 Command-line interface (main.py)

`main.py` implements the interactive loop shown in Listing 8. Commands are explicit slash-commands; any other input is treated as a question.

Listing 8: CLI control flow (`main.py`, excerpt).

```

1 store = LocalVectorStore()
2 show_sources = False
3
4 while True:
5     user_input = input("\nYou> ").strip()
6     if user_input == "/quit":
7         break
8     if user_input == "/sources":
9         show_sources = not show_sources
10        continue
11    if user_input == "/reset":
12        store.reset()
13        continue
14    if user_input == "/ingest":
15        rows = build_local_corpus()
16        if not rows:
17            print("Ingestion produced 0 chunks. Likely Wikipedia rate-limit (HTTP 429).")
18            continue
19        store.upsert_rows(rows)
20        continue
21
22    context, sources = retrieve_context(store, user_input)
23    answer = generate_answer(user_input, context)
24    print(f"Assistant> {answer}")
25    if show_sources and sources:
26        ...

```

13 One-shot ingestion script (ingest.py)

For reproducible demos, `ingest.py` performs a clean sequence: reset the vector store, run verbose ingestion, then upsert all rows. This avoids partially stale Chroma state when re-recording a video.

Listing 9: Batch ingest entrypoint (`ingest.py`).

```
1 def main() -> None:
2     store = LocalVectorStore()
3     print("Resetting vector store before fresh ingest...")
4     store.reset()
5     rows = build_local_corpus(verbose=True)
6     if not rows:
7         print("No chunks were prepared. Retry ingest after a short wait.")
8         return
9     print("\nEmbedding + upsert to Chroma...")
10    store.upsert_rows(rows)
11    print(f"Done. Stored {len(rows)} chunks in vector store.")
```

14 How to run (reproducible checklist)

1. Create and activate a virtual environment.
2. `pip install -r requirements.txt`
3. Install Ollama and pull the configured chat model, e.g. `ollama pull llama3.2:3b`.
4. Run `python ingest.py` (recommended for a clean ingest + upsert), or start `python main.py` and use `/ingest`.
5. Start `python main.py`, optionally enable `/sources`, and ask questions.

15 Design decisions and trade-offs

Option B: single vector store with metadata. Using one Chroma collection with `entity_type` simplifies maintenance and naturally supports mixed queries with a single query path, while still allowing metadata filtering.

Local embeddings vs. Ollama embeddings. The project uses SentenceTransformers for embeddings (homework-allowed) for stable offline behavior and reproducible vector geometry independent of Ollama embedding endpoints.

Rule-based routing vs. learned routing. The classifier is explainable and fast, but can mis-route ambiguous queries; widening filters to both types reduces false negatives at the cost of extra retrieved noise.

LLM grounding vs. comparison questions. Comparison answers depend on whether retrieved chunks jointly contain comparable facts; the system prompt encourages abstaining (`I don't know`) when the context does not support a direct comparison.

16 Limitations and known failure modes

- **Wikipedia rate limits:** aggressive repeated ingestion can yield HTTP 429; the code retries, but the user may still need to wait and re-run.
- **Incomplete ingestion:** if many pages fail, downstream answers may be wrong or empty for missing entities.
- **Chroma disk corruption:** rare but possible; reset/rebuild mitigates it, and SQLite fallback improves availability for Q&A demos.

- **Retrieval quality:** small TOP_K, chunk boundaries, and embedding model capacity limit recall and precision.

17 Conclusion

This project combines retrieval and generation into a single local-first assistant aligned with the homework constraints: Wikipedia ingestion, chunking, local embeddings, metadata-aware vector search, simple query routing, and Ollama-based grounded answers with an explicit unknown response policy. The modular layout maps cleanly to the RAG lifecycle and is documented here with representative code excerpts illustrating the runtime behavior.

This report describes the repository at the time of writing; behavior should always be verified against the latest source code.